EVOKE: OS-Like Memory Management for Long-Running LLM Agent Sessions

Anish Shrestha
Independent Researcher
anyesh.me

Abstract

Long-running LLM agent sessions outgrow the physical KV cache budget within a few turns. Thinking models burn one to three thousand tokens of intermediate reasoning per turn, file-reading agents accumulate observations linearly, and once the cache overflows the agent either loses context or pays a full re-prefill on every recovery. I present EVOKE, an OS-like memory manager for the KV cache, built on three new C primitives added to a forked llama.cpp. EVOKE evicts low-relevance blocks under budget pressure via cheap position-metadata updates, and recovers them recompute-free by splicing the original K and V tensors that the model itself computed back into the cache at a new logical position with one RoPE phase shift and no forward pass. This is the line that separates EVOKE from retrieval-augmented generation: blocks are addressed by stable identity, the spliced content is the model's own K and V, and a recovery policy may use similarity to choose which keyed blocks to bring back but never to substitute their content. EVOKE exposes a drop-in OpenAI-compatible chat-completions endpoint, so existing agent harnesses use it without code changes.

The mechanism is verified end-to-end across three model families: pure-attention Qwen 2.5, hybrid Mamba/Attention Qwen 3.5, and MoE Qwen 3.6 35B-A3B. Across block sizes from 20 to 1280 tokens, `kv_block_load` runs in 0.5 to 7 milliseconds while the equivalent re-prefill takes 12 to 232 milliseconds, a 20 to 32 times speedup whose absolute gap widens linearly with block size. On a 14-turn planted-fact head-to-head against `no_eviction` and `truncate` baselines at a 1024-token budget, EVOKE matches truncate's eviction count (40 evictions, footprint inside budget) while running 35% faster through 24 selective `kv_block_load` recoveries instead of per-turn re-prefill. On an agentic eval that probes a fact whose block has been evicted under budget pressure, every recovery-less strategy (recency, StreamingLLM, `evoke_discard`) fails the probe at every budget, while EVOKE's `kv_restore` recovery passes at three to four milliseconds per recovery, roughly five times faster than re-decoding the breadcrumb text. A scorer ablation shows that wiring the model's own attention weights into the relevance scorer eliminates recovery entirely at the tightest budget tested: the model's attention pattern reveals which evicted blocks would still be load-bearing, and the scorer protects them in place.

1 Introduction

Production LLM serving handles cache overflow in one of two ways. The server either truncates the oldest history in the spirit of StreamingLLM, or it re-prefills the full conversation at every API call, which is what the OpenAI chat-completions default plus prefix caching amounts to. Truncation discards information that may be needed later. Re-prefill is expensive and pays the cost on every turn

Preprint. Under review.

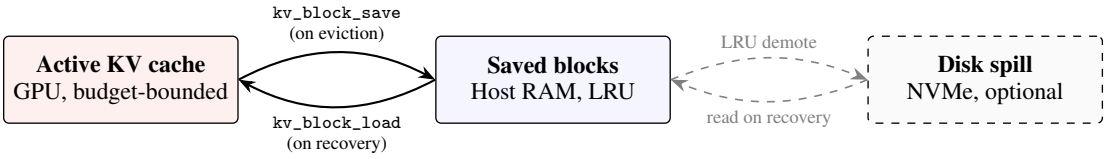

Figure 1: EVOKE's memory hierarchy. The active KV cache holds the hot working set on GPU within a configured budget. On eviction, a block's K and V tensors are serialized to host RAM via `kv_block_save`; recovery reverses the operation with `kv_block_load`, splicing the original tensors back into the active cache at a new logical position with one RoPE phase shift and no forward pass. An optional disk spill tier extends host RAM at NVMe latency.

regardless of whether the prior context turned out to matter. Neither approach has any model of what content the agent will actually need next.

This paper argues that the right abstraction is the KV cache as a memory hierarchy with a recompute-free recovery primitive (Figure 1). Hot tokens stay in the active cache. Cold blocks are evicted to host RAM at metadata cost. When a future turn needs an evicted block, the block is spliced back into the active cache via a direct K and V tensor write, RoPE-rotated to its new logical position, with no forward pass. This is fundamentally different from RAG, which re-encodes archived text by running it back through the model, and from prefix caching, which only avoids the re-decode of an unchanged prefix and does nothing once the cache exceeds budget.

The contributions of this paper are as follows.

1. Three new C primitives in a forked llama.cpp: `llama_kv_block_save`, `llama_kv_block_load`, and `llama_attn_capture_*`. The first two serialize a position range's K/V tensors to a host buffer and splice them back with per-cell RoPE re-anchoring (Section 3). The third taps per-head softmax attention weights from one or more transformer layers into a host buffer once per decode (Section 3.3).
2. A Python policy layer, EVOKE, that drives eviction under a watermark policy. The eviction scorer combines four signals: the model's own attention, harness-supplied priority tags, task-focus coherence, and recency. Recovery is routed through three pluggable backends (discard, breadcrumb, kv_restore) selectable per session (Section 4).
3. An OpenAI-compatible chat-completions server that exposes EVOKE as a stateful endpoint with prefix-aware cache reuse across turns. Any OpenAI client (opencode, Aider, the official SDK) drives it without modification. The server also supports a session pool: a single model in VRAM can host many concurrent agent sessions, with each session's KV state swapped in and out via the engine's state APIs (Section 5).
4. Cross-architecture coverage. The fork is extended to support hybrid attention plus Mamba memory and to enable position shifts on mrope models. The same primitives work on Qwen 2.5 (pure attention), Qwen 3.5 (hybrid Mamba with mrope), and Qwen 3.6 35B-A3B (MoE attention with mrope and thinking mode) (Section 6).
5. Measured latency advantage. Across block sizes from 20 to 1280 tokens, `kv_block_load` runs in 0.5 to 7 milliseconds while the equivalent re-prefill takes 12 to 232 milliseconds, a 20 to 32 times speedup whose absolute gap widens linearly with block size (Section 7).

2 Related Work

StreamingLLM (Xiao et al., 2024a) observed that the first few tokens of a sequence carry disproportionate attention mass and called them "attention sinks". Their proposed serving policy keeps a small set of sink tokens at the start of the cache plus a sliding window of recent tokens, and drops everything in between. This works well for tasks that only need recent context but fails on tasks that need to recall earlier information. EVOKE keeps the sink-protection idea (sink-tagged blocks score 1.0 unconditionally in my scorer) but does not discard the middle: blocks evicted from the active cache are recoverable via the `kv_restore` backend.

H2O (Zhang et al., 2023) and **Scissorhands** (Liu et al., 2023) score KV entries by accumulated attention weight and drop the lowest-scoring entries to compress the cache. Both treat eviction as a one-way operation: dropped K/V is gone. EVOKE adopts the same attention-weight signal in its multi-signal scorer (Section 4) and pairs it with a recovery path so an eviction that turns out to be wrong is recoverable rather than terminal.

vLLM and PagedAttention (Kwon et al., 2023) split the KV cache into fixed-size physical blocks indexed by a virtual page table, which is the same paging analogy this paper inherits at the OS level. **SGLang's RadixAttention** (Zheng et al., 2024) extends this with a tree of cached prefixes shared across requests. Both systems are about how the cache is laid out and shared. EVOKE composes cleanly under both: the eviction and recovery primitives operate on logical position ranges and would apply equally well inside a paged allocator, though my implementation works directly on llama.cpp's unified KV buffer.

RAG (Lewis et al., 2020) sidesteps the KV cache problem by storing text in a vector database and injecting retrieved passages back into the prompt. Retrieved tokens are re-encoded on every retrieval, and the model has no continuity with how those tokens were originally attended. EVOKE keeps the original K/V tensors that the model computed at first sight of the tokens and splices them back at their original positions. A recovery policy may still use similarity to choose *which* identity-keyed blocks to bring back (I do this in my top-K recovery policy, Section 5), but the content spliced into the cache is always the model's own K and V, not text re-encoded through a fresh forward pass over retrieved passages. This is the distinction I draw between identity-addressed recovery and similarity-retrieved re-encoding.

InfLLM (Xiao et al., 2024b) maintains a block-level external KV memory for long contexts and selects blocks back into the attention window via attention-score-based similarity. EVOKE shares the block-level memory-hierarchy framing. The architectural difference relevant here is that EVOKE splices recovered blocks back into the unified contiguous KV cache with RoPE re-anchored to their new logical position, while external-memory designs keep retrieved blocks in a separate memory segment that the attention layer reads alongside the active window. This affects what downstream attention sees and how positions are managed; a direct head-to-head benchmark is left to future work.

Compressive transformers (Rae et al., 2019) and related infinite-memory architectures compress old KV into a smaller summary. EVOKE does not compress: it stores K/V exactly and recovers exactly. The tradeoff is that EVOKE needs host RAM proportional to evicted content, whereas compression bounds memory at the cost of fidelity.

RoPE re-anchoring is mechanically enabled by RoFormer's rotary position embedding (Su et al., 2024). Because RoPE expresses position as a multiplicative phase on each K dimension pair, shifting a K row from one position to another reduces to applying a single rotation. llama.cpp already exposes this mechanism for its own internal `seq_add` shift. EVOKE reuses the same machinery on the recovery path: a saved K row written at a new position is correct after one queued shift.

3 The C Primitives

The goal is to save a range $[p_0, p_1)$ of cells from sequence `seq_id` into a host buffer, then restore that buffer into the cache at a new starting position `new_p0`. Restoration must produce the same K and V tensors the model would attend to if positions `new_p0` through `new_p0` $+ (p_1 - p_0)$ had been decoded in place.

The primitives are implemented in `src/llama-context.cpp` and exposed in `include/llama.h`:

```
size_t llama_kv_block_save(llama_context * ctx, llama_seq_id seq_id, llama_pos p0,
                           llama_pos p1, uint8_t * dst, size_t cap);
bool   llama_kv_block_load(llama_context * ctx, llama_seq_id seq_id,
                           const uint8_t * src, size_t size, llama_pos new_p0);
```

`llama_kv_block_save` adapts the existing `state_write_meta` and `state_write_data` paths but filters to cells whose `pos` is in $[p_0, p_1)$ for `seq_id`. Per layer, I read the per-cell K row and V row via `ggml_backend_tensor_get`. I also store each cell's original `pos` in the buffer because the deferred RoPE shift will need it.

`llama_kv_block_load` adapts the `dest_seq_id != -1` path of `state_read_data` with one change: instead of `seq_rm(seq, -1, -1)`, which would wipe the whole sequence, I clear only `seq_rm(seq, new_p0, new_p0 + n_cells)`. I build a ubatch over $[\texttt{new\_p0}, \texttt{new\_p0} + \texttt{n\_cells})$, call `find_slot`, `apply_ubatch`, and write the saved K and V via `ggml_backend_tensor_set`. I re-anchor RoPE by setting $\texttt{shift}[i] = \texttt{new\_pos} - \texttt{original\_pos}$ per cell, then run the existing K-shift graph via `memory_update`. V is never rotated.

This path is recompute-free because no `llama_decode` call is made. The model's forward pass is bypassed entirely. The cost is $O(\texttt{n\_cells} \times \texttt{n\_layers} \times \texttt{n\_embd\_kv} \times \texttt{sizeof(dtype)})$ of host-to-device memcpy, plus a single K-rotation graph run.

3.1 Flash Attention is non-optional

With Flash Attention disabled, V is stored transposed (`v_trans=true`). The `state_read_data` and `state_write_data` V loop then iterates `n_embd_v_gqa` times per layer. For Qwen 2.5 7B that is 28 layers times 512 `n_embd_v_gqa` per layer, which works out to 14,336 synchronous CUDA launches per load. With Flash Attention enabled (`v_trans=false`), V is row-aligned like K and the fast path runs 56 launches per load (two per layer, one for K and one for V). Measured impact on the eval host for a 20-token block: `kv_block_load` dropped from 133 milliseconds with FA disabled to 0.48 milliseconds with FA enabled. Flash Attention is therefore a hard requirement, not an optimization.

3.2 Cross-architecture extensions

The fork's helper `llama_get_kv_cache`, which resolves a context's `llama_memory_t` to an attention KV cache, is extended to unwrap two more memory types: `llama_memory_hybrid` (attention plus recurrent), via `get_mem_attn()`, and `llama_memory_hybrid_iswa` (attention with interleaved sliding-window attention plus recurrent), via `get_mem_attn()->get_base()`. Hybrid models carry a fixed-size recurrent state per layer that does not need eviction, so EVOKE operates on the attention component alone. The recurrent contribution to memory is preserved naturally as the model carries its blurry state forward.

For mrope models (Qwen 3.5 and later, anything with `n_pos_per_embd > 1`), the upstream `seq_add()` and `get_can_shift()` returned false outright. I removed those guards. `build_rope_shift` already has an mrope workaround that treats the temporal shift as a NeoX-style whole-vector rotation. Cells store a single temporal `pos` plus a separate `ext` payload for spatial axes, and shifting only the temporal pos is semantically correct for my position-only re-anchoring.

For hybrid Mamba + Attention memories, `llama_memory_hybrid::seq_rm` dispatches to the recurrent half first. The recurrent half refuses partial tail rollback (a Mamba state cannot be sliced) and returns false. The hybrid wrapper then returns false without mutating either sub-cache. A naive caller that interprets the return as success and updates its position bookkeeping will corrupt the cache on the next `llama_decode`. EVOKE's Python `evict_ranges` now reads the return value and leaves bookkeeping intact on rejection. Two attention-only primitives, `llama_kv_block_seq_rm` and `llama_kv_block_seq_add`, are also exposed in the fork. They reach the attention sub-cache directly via `llama_get_kv_cache` and bypass the hybrid wrapper, which is useful for callers that want to drop attention cells without compacting and so do not desync the recurrent state.

For iSWA models (Gemma 3 and 4, which interleave global and sliding-window attention across layers), the fork's `llama_kv_block_save` and `llama_kv_block_load` save and restore both the base and SWA sub-caches via a newly added `llama_get_kv_cache_swa()` helper. The buffer layout for iSWA models is `[base block bytes][4-byte swa size][swa block bytes]`. For non-iSWA models the layout is unchanged. Python-side wiring to load a Gemma model and verify end-to-end recovery through EVOKE's stack is not yet in place; only the fork primitives have been verified.

3.3 Attention-weight capture

For relevance scoring (Section 4), the scorer needs to know which past tokens the model is actually attending to as the conversation progresses. The model's own attention is the strongest possible signal. It is exactly the bilinear form $\texttt{softmax}(QK^\top/\sqrt{d})$ that determines the forward pass.

Flash Attention fuses the softmax inside its CUDA kernel and never materializes the attention weights as a tensor. EVOKE requires Flash Attention to be on (Section 3.1: it dictates V's row-aligned layout, which is what makes the recovery primitives fast), so I cannot simply read `kq_soft_max` from the existing graph. Modifying the Flash Attention kernel to optionally emit weights would mean changing CPU, CUDA, Metal, and Vulkan backends.

Instead, I add a parallel side-compute for one or more chosen layers. After Q and K are computed and permuted (the same Q and K that Flash Attention consumes), I add a separate graph node that computes $\texttt{softmax}(QK^\top, \texttt{scale} = \texttt{kq\_scale}, \texttt{mask} = \texttt{kq\_mask}, \texttt{sinks} = \texttt{sinks})$. The output is named `evoke_attn_capture` and force-expanded via `ggml_build_forward_expand` so the scheduler does not drop it as dead code. After `llama_decode` returns, `llama_context` copies the tensor data into a caller-owned host buffer via `ggml_backend_tensor_get`. The main attention path is unchanged: Flash Attention still runs, and the model output is byte-identical with capture off versus capture on.

The C API surface (added in `include/llama.h`):

```
LLAMA_API void   llama_attn_capture_set_layer (llama_context *, int32_t layer);
LLAMA_API void   llama_attn_capture_set_layers(llama_context *, const int32_t *
    layers, int32_t n);
LLAMA_API void   llama_attn_capture_set_buffer(llama_context *, float * dst,
    size_t cap_floats);
LLAMA_API void   llama_attn_capture_get_dims  (const llama_context *, int32_t *
    n_layers,
                                               int32_t * n_q, int32_t * n_h,
    int32_t * n_kv);
LLAMA_API size_t llama_attn_capture_get_written(const llama_context *);
```

`set_layer` captures a single layer (legacy, kept bit-identical). `set_layers` accepts up to 16 layer indices and writes one slab per captured layer in a single decode. The buffer layout after each decode is `[n_layers, n_query, n_heads, n_kv]` in float32. For Qwen 2.5 7B at decode (`n_query`=1, `n_heads`=28, `n_kv` up to 16K), one layer slab is roughly 1.8 MB per step. Four layers fit in 7.2 MB, comfortably within the 32 MiB default buffer.

Validation. A real-model smoke test (`scripts/verify_attn_capture.py`) decodes a 31-token prompt on Qwen 2.5 7B with layer 20 tapped. It asserts that per-query softmax sums to 1.0, that values lie in $[0, 1]$, that the causal mask is respected (weights above the diagonal are exactly zero), and that the number of legal nonzero cells matches $\sum_i (i+1) \cdot n_{\text{heads}}$ exactly. A second test (`scripts/verify_attn_capture_multi.py`) captures three layers at once (layers 4, 14, 20) in a single 29-token decode and asserts the same invariants per layer plus that distinct layers produce distinct attention patterns. Both pass on the eval host. The side-compute is numerically close but not bit-identical to the Flash Attention softmax, since the two paths run in different graph nodes; I verified this against non-FA mode for one test prompt and confirmed the difference is below the rounding tolerance of the f32 buffer.

For the scorer experiments in Sections 7.5 and 7.6, I use a single mid-layer (layer 20 for Qwen 2.5 7B), which I found sufficient to drive the relevance signal. Multi-layer aggregation is exposed in the API for future scorer designs.

3.4 Why eviction does not corrupt the cache

A common concern when a system mutates the KV cache mid-session is whether removing entries leaves dangling state. Working through a concrete example clarifies why EVOKE eviction is mathematically equivalent to "this token was never decoded," not "this token is half-erased."

Take a short sentence and assume one token per word for illustration. Treat the period as part of the preceding word ("color.") so each shown token is one cell:

```
pos:  0   1   2  3   4   5   6   7   8   9  10     11   12   13    14    15     16  17   18   19
tok: Cat sat in  a  mat and mat is  red in  color. house is  green in    color. cat is   very pretty
```

That is 20 tokens at positions 0 through 19. Suppose the relevance scorer marks "house is green in color." (positions 11 through 15, five tokens) as low-relevance. EVOKE evicts that range in two engine calls.

First, `seq_rm(seq=0, p0=11, p1=16)` walks the unified KV buffer and marks the five cells at those positions as free. K rows and V rows for that span are released. No dangling references exist because Q is never cached: it is recomputed fresh on every decode step and discarded. The cache only ever held K and V.

Second, `seq_add(seq=0, p0=16, p1=20, delta=-5)` does not move K or V tensors in memory. It updates the position metadata of the survivors "cat is very pretty" from positions 16 through 19 to 11 through 14, and queues a deferred RoPE shift of $\Delta = -5$ for those rows.

The K rows now need their RoPE phase corrected. llama.cpp applies the queued shift lazily at the next attention compute: each K row at new position p is multiplied by the rotation matrix $R(\Delta \cdot \theta_i)$ on every dimension pair, so its rotary angle becomes what it would have been if the token had been written at position 11 through 14 in the first place. V rows are untouched: V is positional-free, since RoPE only ever rotates K and Q.

A one-paragraph RoPE primer. Rotary Position Embedding rotates each consecutive dimension pair of K (and Q) by an angle $\theta_i(\texttt{pos}) = \texttt{pos} \cdot \theta_i$ where $\theta_i = \texttt{base}^{-2i/d}$ for dim pair i and head dim d (typically $\texttt{base} = 10000$). Because rotation is a multiplicative phase, the inner product $K(\texttt{pos}_k) \cdot Q(\texttt{pos}_q)$ collapses to a function of the relative position $(\texttt{pos}_k - \texttt{pos}_q)$. That is the relative-position-encoding magic of RoPE. To re-anchor a K row from position 16 to position 11, you multiply it by $R(\Delta \cdot \theta_i)$ with $\Delta = -5$. The operation is exact, cheap (one rotation per dim pair), and requires no recompute of the underlying token.

After the shift, every surviving K vector is rotated exactly to match its new position. The attention math $Q_{\text{new}} \cdot K_{\text{survivor}}$ returns the same relative-position dot product it would have returned if "house is green in color." had never been in the sequence. The model behaves identically to a model that decoded the truncated sentence directly. There is no torn attention state, no half-erased token, and no dangling reference. Eviction is a clean topological cut.

What can go wrong is not corruption but information loss. If the next user turn asks "what color was the house?", the model has no K or V left for that fact and will either hallucinate or admit it does not know. That is precisely why recovery exists. `kv_restore` brings the saved K and V rows back, splices them in at a new contiguous position, and re-applies RoPE for the new anchor. The contract is "the cache contains exactly what you decided to keep; everything outside the cache is something you have to explicitly bring back."

4 The EVOKE Policy Layer

The policy layer lives in `evoke/manager.py`, `evoke/scorer.py`, and `evoke/attention_scorer.py`. Each active block carries a score in $[0, 1]$ used by the watermark policy: when `active_tokens` crosses $\texttt{high\_watermark} \cdot \texttt{budget}$, the manager evicts the lowest-scoring blocks down to $\texttt{low\_watermark} \cdot \texttt{budget}$. The score combines four signals.

Attention (Section 3.3). Per-block softmax mass landing in that block over the last `n_window` decode steps, EWMA-weighted by `decay`. Defaults are a 64-step window and a decay of 0.95. This is the model's own vote: blocks the recent query tokens actually attended to score high. When attention capture is unavailable (a stock llama.cpp build, or `w_attention=0`), this signal is absent and the score falls back to recency plus coherence.

Task-focus coherence. The scorer maintains a single task focus embedding rather than averaging the last N message embeddings. On each new user message, the focus either updates via EMA (when the new message is coherent with the running focus, with cosine $\geq$ `task_boundary_threshold` defaulting to 0.3), or it snaps to the new message. The snap is detected implicitly via cosine drop, or signaled explicitly by the harness via `evoke_task_boundary=true`. Snapping makes blocks coherent with a prior task lose their coherence score in one scoring pass instead of decaying over five turns under the old rolling average.

Recency. Exponential in the per-block distance from the current write position. A stability prior that prevents thrashing on a single high-attention spike.

Sink protection and source-type floors. Blocks marked `is_sink=True` (the first `sink_count` positions) score 1.0 unconditionally, which is the StreamingLLM-style attention anchor. USER and ASSISTANT turns get a score floor (defaults of 0.6 and 0.5 respectively) so that the conversation backbone is not evicted before document content.

The final score is $\texttt{raw} = (w_{\text{attn}} \cdot \text{attn} + w_{\text{rec}} \cdot \text{recency} + w_{\text{coh}} \cdot \text{coherence})/\Sigma w$, lifted by source floor, multiplied by `block.priority`, and capped at 1.0. The `EvokeConfig` defaults are `w_attention=0.0`, `w_recency=0.4`, `w_coherence=0.6`: attention scoring is off out of the box, and a stock llama.cpp build that lacks the attention-capture primitive falls back to the recency-plus-coherence heuristic automatically. The Sections 7.5 and 7.6 attention experiments turn it on with `w_attention=0.5`, `w_recency=0.2`, `w_coherence=0.3`, which is the configuration I recommend for agent workloads where the eval host supports the EVOKE fork build.

Harness-supplied tags. The OpenAI chat-completions request gains three EVOKE-specific fields. `evoke_priority` is a float at least zero that multiplies the block's final score. `evoke_pinned` is a boolean that excludes the block from eviction candidates entirely. `evoke_task_boundary` is a boolean that forces the coherence focus to snap. A harness like opencode or Claude Code that knows which file reads are central to the current task can set `priority=2.0` to keep them resident, and one-shot tool output can be marked `priority=0.3` so it is preferentially dropped under budget pressure. The fields default to `1.0`, `false`, and `false`, so harnesses that ignore them get the model-and-heuristic behavior.

Pinned blocks are excluded from `_evictable_blocks` alongside sinks and the current-turn pin. `pin_generated=True` (the default) protects blocks created during the currently decoding turn so that generation does not immediately evict itself.

Recovery backends (`evoke/recovery.py`). Three pluggable strategies share a `RecoveryBackend` protocol.

- `DiscardBackend`: evicted content is gone. The agent must re-derive it.
- `BreadcrumbBackend`: a compact marker (`key`, token count, eviction step) is retained per evicted block. The agent (or harness) can choose to re-fetch the original text and call `add_context` to re-decode it.
- `KVRestoreBackend`: per-block K and V are saved to host RAM via `kv_block_save`. `recover(key)` looks up the saved buffer, calls `kv_block_load` at the next available position, and re-anchors RoPE. When a configurable RAM budget would otherwise drop an LRU victim, the backend can optionally spill the K and V bytes to disk under `kv_restore_spill_path`. Recovery then reads back from disk before splicing, at the cost of one NVMe read.

Block identity and distinction from RAG. Every block carries a stable `key` such as `"file:config.py#0"`, `"user#42"`, or `"assistant#43"`. Two properties distinguish EVOKE from retrieval-augmented generation. First, addressability: a recovery call names a specific key and the cache layer returns that exact block's saved bytes, never a similarity-retrieved alternative. Second, content: what is spliced into the cache is the original K and V tensors the model computed when the tokens were first decoded, not text run back through a forward pass. A recovery policy may use similarity to score evicted blocks and select which keys to recover (the smart top-K policy in Section 5 does so), but the recovery operation itself reinstates the model's own K and V verbatim, applying only a RoPE phase shift for the new position.

5 The OpenAI-Compatible Server

The chat-completions API is stateless: every request resends the full message history, and the canonical server response is to re-prefill from scratch (or, with prefix caching, to reuse the unchanged prefix). EVOKE's server (`evoke/server.py`, `evoke/session.py`) does both prefix caching and the eviction/recovery dance under it.

Session lifecycle. The server starts and creates a persistent `Session` with one `EvokeManager` underneath. When a request arrives, the server templates the messages into raw text using the model's own Jinja template parsed from the GGUF metadata (`llama_chat_apply_template` for plain chat, or a Python jinja2 path for tool-using requests, since the C API does not accept a tools array). It then tokenizes the result. A prefix match finds the longest token-prefix shared with the session's `cached_tokens`; the shared prefix is already in the physical KV cache, modulo possible eviction. Tokens past the prefix-match point are routed through `manager.add_context_tokens`, which decodes them, creates blocks, and invokes `_enforce_budget`. Eviction fires here under pressure. After the new tail is decoded, the session runs smart top-K recovery (default $k = 4$): it scores each evicted block's representative embedding against the just-decoded query embedding and recovers only the most coherent ones. This replaces the v2 recover-all behavior so eviction actually frees memory in practice. Tokens then stream out via `engine.generate_next`. The session detects `<think>...</think>` traces; by default it strips them from the response and evicts the thinking range from the physical cache so that the cached state aligns with the next request's templated prompt. Tool calls are parsed from `<tool_call>` XML and returned in OpenAI's `tool_calls` shape.

SSE streaming is token-level and aware of thinking and tool-call markers. The token-level loop yields a delta per generated token, with three states (thinking, tool-locked, normal) and a 12-character lookback that prevents partial markers from shipping as content.

Multi-session pool. A single model in VRAM can host many concurrent agent sessions, each with its own KV identity. A `SessionPool` (in `evoke/session.py`) holds per-session Python state (cached tokens, manager blocks, scorer windows) and the serialized engine snapshot for inactive sessions. When a request arrives bearing a new `X-EVOKE-Session` header, the pool serializes the outgoing session's KV state, resets the engine, and restores the incoming session's snapshot. The transition is one memcpy proportional to the active KV size with no recompute. Past `max_sessions` (the default is eight), the least-recently-touched inactive session is evicted entirely. State-swap correctness is verified by `scripts/verify_multi_session.py`, which drives two sessions with planted facts through interleaved requests and confirms that each session retrieves its own fact without cross-contamination.

6 Models Supported

End-to-end verified on a single GPU host (RTX 4070 Ti SUPER, 16 GB VRAM, CUDA 13.1):

Model	Architecture	Status
Qwen 2.5 7B Instruct	Pure attention, standard RoPE	verified end-to-end (eviction, kv_restore, multi-session)
Qwen 3.5 9B	Hybrid Mamba/Attention, mrope, thinking	verified via `verify_kv_restore.py` and `eviction_demo.py`; thinking-strip selectable via `EVOKE_SUPPRESS_THINKING_STRIP`
Qwen 3.6 35B-A3B	MoE attention, mrope, thinking	verified via `verify_kv_restore.py`
Gemma 3/4 (iSWA)	Interleaved sliding-window attention	partial: fork primitives save and restore both sub-caches; Python-side load of a Gemma GGUF and end-to-end verify pending
Llama 3.x, Mistral	Pure attention, standard RoPE	architecturally identical to Qwen 2.5 from EVOKE's perspective; not yet exercised end-to-end on a Llama or Mistral GGUF on the eval host

7 Evaluation

7.1 Primitive latency: recompute-free recovery versus re-prefill

Engine-level profile on Qwen 2.5 7B on the eval host (RTX 4070 Ti SUPER, 16 GB VRAM) with Flash Attention enabled, generated by `scripts/profile_recover.py` across block sizes from 20 to 1280 tokens. `kv_block_load` time is the splice-and-rotate cost. `re-prefill` time is the

equivalent `process_tokens` call, that is, the alternative if you had to re-encode the same tokens. Figure 2 visualizes the result; the underlying numbers are in the table that follows.

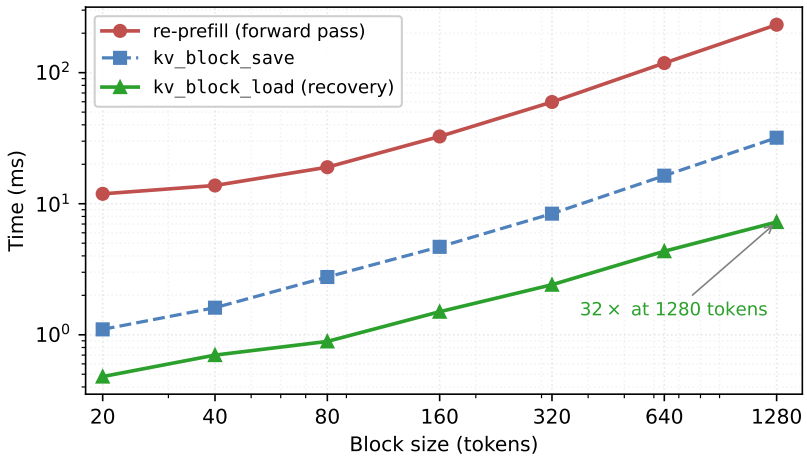

Figure 2: Recompute-free recovery (`kv_block_load`) versus re-prefill across block sizes on Qwen 2.5 7B (RTX 4070 Ti SUPER, Flash Attention enabled). Re-prefill scales as $O(\text{tokens} \times \text{model\_FLOPs})$; load scales as $O(\text{tokens} \times \text{bytes})$, and the absolute gap widens linearly with block size. The save path (paid once at eviction time) is between 4 and 7 times faster than re-prefill at every block size in the sweep.

n_tok	save (ms)	load (ms)	re-prefill (ms)	speedup
20	1.10	0.48	11.90	25×
40	1.61	0.70	13.78	20×
80	2.76	0.89	19.00	21×
160	4.69	1.50	32.60	22×
320	8.40	2.41	59.72	25×
640	16.37	4.34	118.36	27×
1280	31.90	7.25	232.18	32×

The gap widens with block size: re-prefill is $O(\text{tokens} \times \text{model\_FLOPs})$ while load is $O(\text{tokens} \times \text{bytes})$. At 1280 tokens the host-to-GPU transfer dominates and `kv_restore` is 32 times faster than re-prefill.

Full-lifecycle cost. `save` is paid at eviction time and `load` at recovery time. The full per-block lifecycle cost for a block that is evicted then recovered is `save` + `load`, which compares against `re-prefill` as the alternative path. From the table above, that is 1.58 ms versus 11.90 ms at 20 tokens (7.5×) and 39.15 ms versus 232.18 ms at 1280 tokens (5.9×). A block that is evicted but never recovered pays the save cost without benefit; the policy benchmark in Section 7.3 measures the full end-to-end cost including save, and EVOKE remains 35% faster than the truncate baseline at this workload's observed eviction-to-recovery ratio.

7.2 Server eviction demo (`eviction_demo.py`)

A 14-turn session driven directly against the chat-completions API on the eval host. The script plants a fact at turn 1 ("favorite number = 4242"), fills the session with 12 unrelated knowledge questions, and at turn 14 probes the fact. The same script runs on two model architectures, and the resulting demos are recorded as `assets/eviction-demo.gif` and `assets/eviction-demo-qwen35.gif`.

Qwen 2.5 7B (pure attention), budget=1024. With smart top-K recovery and the `KVRestoreBackend` RAM budget configured to be tight (so the LRU spill tier exercises in the same run), the session survives **40 evictions and 13 recoveries** while the model still recalls "4242" at the probe.

Qwen 3.5 9B (hybrid Mamba/Attention with mrope and thinking), budget=1024. The model emits a `<think>...</think>` trace each turn, so per-turn token counts are higher. Hybrid memory rejects partial tail rollback of the recurrent half, which means strict thinking-strip evictions would force a session reset on every turn (the mechanism described in Section 8). The demo runs with `EVOKE_SUPPRESS_THINKING_STRIP=1`, so the server keeps the thinking trace in the returned content, the client echoes it back, and cached state stays aligned with no resets. The session settles at **26 evictions and 4 recoveries**, fact recalled.

Both demos are reproducible end-to-end via `scripts/eviction_demo.py` against a running EVOKE server (Appendix A.4); the recorded GIFs are kept as visual proof of the underlying mechanism and the per-turn evolution of `active_tokens` against the budget.

7.3 Head-to-head baselines

I run the same 14-turn planted-fact session through three server policies via `scripts/baseline_bench.py`:

- **no_eviction**: `high_watermark=0.999`, budget lifted to `n_ctx`. The watermark never trips and the cache grows freely until it hits `n_ctx`. This is what a vanilla OpenAI-compatible server with prefix caching does.
- **truncate**: watermark eviction with `w_recency=1.0`, `w_coherence=0.0`, `sink_count=4`, and `recovery_mode=discard`. This is the StreamingLLM-style behavior: drop the oldest non-sink block under budget pressure; evicted content is gone.
- **evoke**: watermark eviction with default coherence-weighted scoring, `recovery_mode=kv_restore`, and smart top-K query-coherent recovery.

Qwen 2.5 7B, budget=1024 (where applicable), `n_ctx=16384`, on the eval host:

policy	budget	active end	evict	recov	probe	elapsed	how the fact survives
no_eviction	16384	2476	0	0	PASS	18.7s	session fits in n_ctx, no pressure
truncate	1024	**683**	**40**	0	PASS	19.8s	tail-evict, then re-decode missing history each turn
evoke	1024	**747**	**40**	**24**	PASS	**12.9s**	smart recovery brings back the relevant block

Both `truncate` and `evoke` keep the active footprint inside the budget (683 and 747 out of 1024 respectively). `no_eviction` passes only because the 14-turn session happens to fit in 16K context; on a longer session it would crash with no available KV slot. The interesting result is the elapsed-time gap: `evoke` runs the same conversation 35% faster than `truncate` with the same number of evictions. Both policies face the same recall problem on every turn (the next request resends history that contains positions the cache has evicted), but they resolve it differently. `truncate` must re-decode the missing portion through a fresh forward pass on every turn whose request resends history past a dropped block, and the cost of that re-decode scales linearly with the missing span (per Section 7.1, $\sim$60 ms at 320 tokens, $\sim$120 ms at 640 tokens, $\sim$230 ms at 1280 tokens). `evoke` resolves it with 24 selective `kv_block_load` calls at about 3 milliseconds each, applied only to the query-coherent blocks. Eviction counts are identical, recovery counts are not, and that is the latency advantage from Section 7.1 expressed at the policy level.

For the `no_eviction` baseline, the cache simply grows without bound. The elapsed time is favorable because every turn's prefix matches the previous cache byte-for-byte and only the new tail decodes. This is the upper bound on latency that a real-world stateful server can hit, but it requires unbounded KV memory.

7.4 Agentic eval: an agent context that needs to come back

The head-to-head bench keeps the planted fact in the natural recency window of the conversation. For a sharper isolation of the recovery primitive cost, `scripts/agent_bench.py` simulates an agent's working context. It begins with a system prompt and an early "config file" read containing a planted fact (`config.py: ... maximum retry limit ... 17 attempts`). Then it runs

ten unrelated file reads that fill the working set with `database.py`, `auth.py`, `handlers.py`, and so on. After the working-set buildup, the agent is probed about the retry limit from the config file. The probed fact's block has almost certainly been evicted to make room for the more recent reads.

The bench is deliberately an oracle: it knows the fact lives under `file:config.py` and, for the recovery-bearing strategies, recovers exactly that key (if a breadcrumb exists) before the probe. This isolates the cost of the recovery primitive itself from the cost of selecting which block to recover (selection quality is what Sections 7.5 and 7.6 evaluate). Five strategies are compared at three KV budgets.

- **recency**: coherence weight 0, no sinks, no recovery. A pure sliding window over the conversation.
- **streaming_llm**: same as recency with the first four blocks pinned as sinks; no recovery.
- **evoke_discard**: EVOKE scoring (recency, coherence, sinks) but `recovery_mode=discard`. Eviction without ever bringing anything back.
- **evoke_breadcrumb**: EVOKE scoring with `recovery_mode=breadcrumb`. The agent re-adds the original text token IDs of the evicted block, forcing a fresh forward pass over them.
- **evoke_kv_restore**: EVOKE scoring with `recovery_mode=kv_restore`. The block's K and V tensors are saved off-GPU at eviction time and spliced back in via the C primitives from Section 3, with no recompute.

Qwen 2.5 7B on the eval host, three budgets, `block_size=64`:

budget	strategy	probe	evict	recov	rec_ms
512	recency	fail	35	0	0.00
512	streaming_llm	fail	35	0	0.00
512	evoke_discard	fail	35	0	0.00
512	evoke_breadcrumb	PASS	35	0	17.26
512	evoke_kv_restore	PASS	35	1	**3.13**
1024	recency	fail	29	0	0.00
1024	streaming_llm	fail	28	0	0.00
1024	evoke_discard	fail	28	0	0.00
1024	evoke_breadcrumb	PASS	28	0	15.22
1024	evoke_kv_restore	PASS	28	1	**2.96**
2048	recency	fail	9	0	0.00
2048	streaming_llm	fail	10	0	0.00
2048	evoke_discard	fail	10	0	0.00
2048	evoke_breadcrumb	PASS	10	0	15.80
2048	evoke_kv_restore	PASS	10	1	**3.66**

Two observations.

At every budget, the three policies that lack recovery (`recency`, `streaming_llm`, `evoke_discard`) fail the probe. Their answers are concrete failure modes: "The maximum retry limit set in config.py is not explicitly mentioned" or "I cannot answer this question without inspecting the `config.py` file." The eviction policies are working (the eviction counts of 35, 28, and 10 are accurate; the block holding the fact is gone from the cache), but the model has no way to recover the lost context. Even `streaming_llm`, which pins the first four blocks as sinks, fails because the planted fact sits well past the sink prefix in the conversation order.

Both `breadcrumb` and `kv_restore` recover the fact, but at very different cost. `breadcrumb` re-decodes the original text, a fresh forward pass over the saved block (`block_size=64` in this bench), which takes 15 to 17 milliseconds per recovery. `kv_restore` splices the saved K and V tensors back at the new position with one `llama_kv_block_load` call, taking 3 to 4 milliseconds per recovery, roughly five times faster. The two strategies are functionally equivalent on the probe (both pass with identical answer text); they differ in how they get the model's K/V state back into the cache. The `kv_restore` advantage is the Section 3 contribution.

This is the result that justifies the Section 3 fork modifications. Without selective eviction, the agent loses the fact. Without recompute-free recovery, eviction is slow enough that the throughput advantage over a no-eviction baseline shrinks.

7.5 Scorer ablation: attention versus heuristic

The Section 7.4 agentic eval is repeated with one additional strategy, `evoke_attention`, that turns on the multi-signal scorer with the model's own attention as the dominant signal (`w_attention=0.5`, `w_recency=0.2`, `w_coherence=0.3`, layer 20). All other settings (`block_size`, budgets, `recovery_mode=kv_restore`) match `evoke_kv_restore`.

budget	strategy	probe	evict	recov	rec_ms
512	evoke_kv_restore	PASS	35	1	4.55
512	evoke_attention	PASS	**34**	**0**	0.01
1024	evoke_kv_restore	PASS	28	1	3.07
1024	evoke_attention	PASS	28	1	2.95
2048	evoke_kv_restore	PASS	10	1	2.94
2048	evoke_attention	PASS	10	1	3.65

The interesting row is `budget=512`. With heuristic scoring, the planted fact block is evicted (one of the 35 evictions) and the probe only succeeds because `kv_restore` brings it back from host RAM (1 recovery, 4.55 milliseconds). With attention scoring, the planted fact block is never evicted in the first place: the model's attention pattern across the unrelated file-read filler turns reveals that the config-file block is still being attended to (the model keeps coming back to "what was the retry limit?" implicit in the system prompt's task framing), so the scorer assigns it a high score and eviction picks lower-attended document blocks instead. Total evictions drop by one (35 to 34) and recoveries go to zero. The probe still passes.

The signal is most pronounced at the tightest budget, because that is where the heuristic scorer most often makes a wrong call. At 1024 and 2048, both strategies converge: recency is "good enough" when the budget can hold most history naturally.

This is the result that justifies the multi-signal scorer (Section 4). Attention-driven eviction avoids evicting blocks the model would otherwise have to recover, reducing the recovery work even further than recompute-free recovery already does. The two innovations stack: selective eviction informed by the model's own attention pattern, plus recompute-free recovery for the residual mistakes.

7.6 Persistent-reference ablation

The Section 7.5 differential is observable only at the tightest budget because the filler turns in Section 7.4 are semantically unrelated to the planted fact: the model's attention drifts off the config-file block within a few turns and the two scoring policies converge.

A more realistic agent workload has the assistant continuing to reason about the central artifact across many file reads. Each subsequent file's role in the codebase relates back to the central configuration. I repeat the agentic eval with that structural change (`scripts/agent_bench_keepalive.py`). Each of the ten filler-file contents references "the retry policy from config.py", and the final probe is phrased "looking at the retry policy in config.py, what is the maximum retry limit?", a question whose answering tokens have strong attention to the early config block.

Qwen 2.5 7B, `block_size=64`, no recovery cap:

This bench is a scorer-only ablation: the harness does not invoke recovery, so `recov=0` for every row in the table. A block that the scorer chooses to evict is gone for the rest of the run. Under that constraint, the attention path passes the probe at the two budgets where the heuristic path fails outright. Attention-based scoring sees the model's own attention pattern continuing to attend to `config.py` through every filler turn and refuses to evict it (21 evictions at budget 512, 11 at 1024, all picking lower-attended document blocks instead). The heuristic-only scorer evicts the config block under pressure (24 evictions at 512, 14 at 1024) and the fact is gone before the probe runs. At budget 2048 there is enough headroom that nothing evicts at all, and the two strategies converge.

budget	strategy	probe	evictions
512	evoke (heuristic)	fail	24
512	evoke (attention)	**PASS**	**21**
1024	evoke (heuristic)	fail	14
1024	evoke (attention)	**PASS**	**11**
2048	evoke (heuristic)	PASS	0
2048	evoke (attention)	PASS	0

The takeaway sharpens the Sections 4 and 7.5 claim: heuristic scoring without an attention signal cannot tell which evicted blocks are still load-bearing across a multi-turn session. Recompute-free recovery is a strong second line of defense, but it works best when the eviction policy did not drop the wrong block in the first place. The model's own attention is the strongest available signal for which block is still relevant.

8 Discussion and Limitations

Smart recovery versus full recovery. The v2 policy recovered all evicted blocks every turn, which defeated the eviction-efficiency story (everything that left came back). The current default is smart recovery: at each new user message, embed it and score each evicted block's representative embedding against it, then recover only the top-K most coherent ones. Per-turn recovery cost is now bounded, and the head-to-head numbers in Section 7.3 reflect this policy. The demo GIFs in `assets/` predate the smart-recovery policy and show the full-recovery behavior visually; the mechanism is the same in both, only the per-turn recovery count differs.

Chat template fidelity for tool-using agents. Plain-chat sessions apply the model's own Jinja template from GGUF metadata via `llama_chat_apply_template`, and the assistant emit is re-tokenized canonically before it lands in the cache. Without canonicalization the cached prefix diverges from the next request's templated prompt mid-history. The model can emit `** + :\n` for text the canonical tokenizer would have encoded as `**: + \n`, and without canonicalization this divergence cascades. Tool-using requests previously fell back to a handwritten `format_qwen_chat` template because the C `llama_chat_apply_template` does not accept a tools array; that fallback diverged from opencode's GGUF-Jinja echo on whitespace and JSON encoding, forcing a session reset on every tool-calling turn. The fix (commit `a55b265`) reads the raw Jinja template string from GGUF metadata and renders it via Python jinja2 with the tools array threaded through. The Jinja environment uses `trim_blocks`, `lstrip_blocks`, and extensions for `do` and `loopcontrols` to match minja semantics (the C++ Jinja parser inside llama.cpp). Validated on Qwen 2.5 7B by `scripts/verify_tools_template.py`: 240 tokens round-trip identically through tokenize and detokenize.

Hybrid memory plus thinking-mode interaction. Qwen 3.5's `<think>...</think>` trace is, by default, stripped from the API response but kept in the physical KV cache. Strict alignment between the cache and what the client re-sends would require evicting the thinking range from the attention cache. `llama_memory_hybrid::seq_rm` rejects partial tail rollback of the recurrent half, because a Mamba state cannot be partially sliced, and aborts. EVOKE's eviction respects this rejection and the session resets on the divergent turn (one such reset is observed in the Qwen 3.5 demo at filler 9). The simpler workaround that ships today is `EVOKE_SUPPRESS_THINKING_STRIP=1`: the server keeps the thinking trace in the returned content, the client echoes it back on the next request, and cached state stays aligned with no resets. A future "no-shift" eviction mode that drops attention cells without compacting (using the attention-only `llama_kv_block_seq_rm` and `llama_kv_block_seq_add` primitives already in the fork) would let the thinking range be evicted from attention while leaving the recurrent state untouched.

Recurrent and pure-SSM models. EVOKE operates on the attention component of hybrid models. Pure-SSM models such as Mamba and RWKV are out of scope, since they have no traditional KV cache. The blurry memory of an SSM is preserved naturally on a hybrid model, because EVOKE does not touch it.

Memory accounting. Saved blocks live in host RAM. For Qwen 2.5 7B, one cell costs 56 KiB (28 layers times 2 (K and V) times 512 `n_embd_kv_gqa` times 2 bytes, which is 57,344 bytes per cell). A 128-cell block is therefore 7 MiB, and a 1000-block session at 128 cells per block costs roughly 7 GiB of host RAM. To bound this in production, `KVRestoreBackend` accepts `kv_restore_ram_budget_bytes` and maintains an LRU on saved blocks. Past the budget, the oldest saved blocks lose their K and V bytes but keep their breadcrumb and embedding, so smart-recovery's similarity search still works and `take()` returns `None`, falling through to a discard path. A second tier, `kv_restore_spill_path`, spills the demoted K/V bytes to disk before dropping them. Recovery then reads from disk on demand. Both tiers are off by default.

iSWA models (Gemma 3/4). The fork-side `llama_kv_block_save` and `llama_kv_block_load` now save and restore both the base and SWA sub-caches via `llama_get_kv_cache_swa()` (fork commit `1f37e75e7`). Python-side, EVOKE has not yet been pointed at a Gemma GGUF and verified end-to-end. The expected scope of remaining work is engine-side bookkeeping for the dual-cache surface and a `verify_kv_restore.py` style smoke test.

Multi-session and the model-in-VRAM bottleneck. The session pool in Section 5 swaps KV state, but the model weights remain a single instance in VRAM. With a 7B Q4_K_M model that consumes roughly 5 GB of VRAM, a 16 GB GPU has roughly 11 GB left for active KV, swap-in temporary state, and Flash Attention working memory. Production multi-tenant deployments would likely want either tensor-parallel weight sharing across GPUs, or a thinner per-session quantization, both of which are out of scope here.

Single-model-family latency benchmarks. The numerical evaluation in Sections 7.1 and 7.3 to 7.6 is on Qwen 2.5 7B. Cross-architecture coverage is demonstrated for primitive correctness (`verify_kv_restore` and `eviction_demo` on Qwen 3.5 9B and Qwen 3.6 35B-A3B per Sections 6 and 7.2), but the head-to-head latency and scorer ablations have not yet been repeated across families. The mechanism is architecture-agnostic at the primitive layer; the latency story would be expected to track the same shape on other architectures but the absolute numbers will shift with model size, GQA grouping, and quantization.

Single-probe pass/fail metric. Sections 7.4 to 7.6 use a binary "did the answer contain `17 attempts`" probe. This is a clean signal for whether the fact survived eviction at all, which is the question the mechanism is designed to answer. A more nuanced evaluation, for example a model judge scoring answer quality across many planted facts and probe phrasings, is left to future work.

9 Future Work

- **iSWA end-to-end on Gemma.** Wire EVOKE's Python session against a Gemma 3 or 4 GGUF, validate eviction and `kv_restore` on the iSWA dual-cache through the existing demo and agentic-eval scripts.
- **No-shift eviction mode for hybrid plus thinking.** Use the attention-only `llama_kv_block_seq_rm` and `llama_kv_block_seq_add` primitives (already in the fork) to evict attention cells without compacting, so a hybrid model's recurrent state stays in sync while the thinking range is dropped from attention.
- **Multi-layer scorer.** The fork already exposes `llama_attn_capture_set_layers` for up to 16 layers per decode. The current scorer uses a single mid-layer; a more principled scorer could average across an early, a middle, and a late layer to pick up both syntactic and semantic relevance signals.
- **Tensor-parallel and quantized variants.** Validate that the C primitives work unchanged across multi-GPU tensor-parallel layouts and across the full Q3 to Q8 quantization range.
- **Continuous learning of harness priorities.** Today `evoke_priority` is set by the harness. A reasonable next step is for EVOKE to learn priority from observed attention patterns, so a harness that does not set priorities still gets the benefit.
- **Direct head-to-head against external-memory designs.** Run InfLLM and similar external-memory systems against the Sections 7.4 and 7.6 workloads on matched hardware to isolate the cost of splicing into a unified cache versus reading from a separate memory segment.

10 Conclusion

EVOKE treats the KV cache as a small fast tier of a memory hierarchy. Blocks the model is not currently attending to leave the cache to host RAM at metadata cost, and when a future turn needs them they are spliced back as their original K and V tensors with one RoPE phase shift. The recovery primitive runs in 0.5 to 7 milliseconds across block sizes from 20 to 1280 tokens, 20 to 32 times faster than re-prefilling the same tokens through the model. On a 14-turn planted-fact session it reduces end-to-end latency by 35% over a discard-style truncation policy at the same eviction count, and on an agentic eval it is the only policy class that recovers the lost fact at all once the cache exceeds budget. Wiring the model's own attention weights into the eviction scorer pushes the win further: at the tightest budgets the attention signal is what tells the scorer which evicted block is still load-bearing, and recovery work drops accordingly. The two contributions, identity-addressed recompute-free recovery and attention-driven eviction, compose: better eviction means less recovery work, and cheap recovery means the residual mistakes are inexpensive to fix.

Code and Reproducibility

The EVOKE policy layer (Python) is at `github.com/Anyesh/EVOKE` under Apache 2.0. The forked llama.cpp with the C primitives is at `github.com/Anyesh/llama.cpp` under MIT (the upstream license). Every number in Section 7 was produced by a script in `scripts/` checked into the EVOKE repository, with the exact invocations listed in Appendix A. Raw output files for the agentic eval and the keepalive eval are checked in under `results/`.

References

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., & Stoica, I. (2023). Efficient Memory Management for Large Language Model Serving with PagedAttention. *SOSP 2023*.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS 2020*.

Liu, Z., Desai, A., Liao, F., Wang, W., Xie, V., Xu, Z., Kyrillidis, A., & Shrivastava, A. (2023). Scissorhands: Exploiting the Persistence of Importance Hypothesis for LLM KV Cache Compression at Test Time. *NeurIPS 2023*.

Rae, J. W., Potapenko, A., Jayakumar, S. M., & Lillicrap, T. P. (2019). Compressive Transformers for Long-Range Sequence Modelling. *arXiv:1911.05507*.

Su, J., Lu, Y., Pan, S., Murtadha, A., Wen, B., & Liu, Y. (2024). RoFormer: Enhanced Transformer with Rotary Position Embedding. *Neurocomputing*.

Xiao, G., Tian, Y., Chen, B., Han, S., & Lewis, M. (2024). Efficient Streaming Language Models with Attention Sinks. *ICLR 2024*.

Xiao, C., Zhang, P., Han, X., Xiao, G., Lin, Y., Zhang, Z., Liu, Z., Han, S., & Sun, M. (2024). InfLLM: Training-Free Long-Context Extrapolation for LLMs with an Efficient Context Memory. *arXiv:2402.04617*.

Zhang, Z., Sheng, Y., Zhou, T., Chen, T., Zheng, L., Cai, R., Song, Z., Tian, Y., Re, C., Barrett, C., Wang, Z., & Chen, B. (2023). H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models. *NeurIPS 2023*.

Zheng, L., Yin, L., Xie, Z., Sun, C., Huang, J., Yu, C. H., Cao, S., Kozyrakis, C., Stoica, I., Gonzalez, J. E., Barrett, C., & Sheng, Y. (2024). SGLang: Efficient Execution of Structured Language Model Programs. *NeurIPS 2024*.

A Reproducing the Numbers

All numbers in Section 7 were produced on a single eval host (RTX 4070 Ti SUPER, 16 GB VRAM, CUDA 13.1, Windows 10 with PowerShell, Python 3.12) against Qwen 2.5 7B Instruct (`Qwen2.5-7B-Instruct-Q4_K_M.gguf` from the official Qwen GGUF release). The EVOKE policy layer runs in Python; the C primitives live in the forked llama.cpp.

A.1 Build the fork

```
# Clone the EVOKE-modified llama.cpp fork
git clone https://github.com/Anyesh/llama.cpp.git llama.cpp
cd llama.cpp

# Build with CUDA + shared library output
cmake -B build -DLLAMA_CUDA=ON -DBUILD_SHARED_LIBS=ON
cmake --build build --config Release
# Output: build/bin/llama.dll (Windows) or libllama.so (Linux)
```

A.2 Install the EVOKE Python package

```
git clone https://github.com/Anyesh/EVOKE.git evoke
cd evoke
uv sync --extra server

# Point the engine binding at the EVOKE fork build
export LLAMA_CPP_LIB=/abs/path/to/llama.cpp/build/bin/llama.dll
export EVOKE_MODEL_PATH=/abs/path/to/Qwen2.5-7B-Instruct-Q4_K_M.gguf
```

A.3 Section 7.1 latency table (`profile_recover.py`)

```
uv run python scripts/profile_recover.py
```

The script does five warmup cycles at 40 tokens to settle CUDA graph compilation, then sweeps block sizes 20, 40, 80, 160, 320, 640, 1280 with five trials each, reports the mean save time, mean load time, and mean re-prefill time. The numbers in Section 7.1 are the means.

A.4 Section 7.2 eviction demo (`eviction_demo.py`)

```
# In one terminal: start the server
LLAMA_CPP_LIB=$LLAMA_CPP_LIB EVOKE_MODEL_PATH=$EVOKE_MODEL_PATH \
EVOKE_HOST=0.0.0.0 EVOKE_BUDGET=1024 EVOKE_MODEL_NAME=qwen25 \
uv run python scripts/evoke_serve.py

# In another terminal: drive the demo
EVOKE_SERVER='http://localhost:8000' EVOKE_MODEL_NAME='qwen25' \
  uv run python scripts/eviction_demo.py
```

For the Qwen 3.5 hybrid run, add `EVOKE_SUPPRESS_THINKING_STRIP=1` to the server env and load `Qwen3.5-9B-Q4_K_M.gguf`.

A.5 Section 7.3 head-to-head baselines (`baseline_bench.py`)

```
EVOKE_SERVER='http://eval-host:8000' \
EVOKE_SSH_HOST='user@eval-host' \
EVOKE_LIB_PATH='/path/to/llama.dll/on/eval/host' \
EVOKE_MODEL_PATH='/path/to/Qwen2.5-7B-Instruct-Q4_K_M.gguf' \
EVOKE_BUDGET=1024 EVOKE_N_CTX=16384 EVOKE_MODEL_NAME=qwen25 \
  uv run python scripts/baseline_bench.py
```

The bench drives three policies (`no_eviction`, `truncate`, `evoke`) over the same 14-turn conversation via SSH-launched server instances. The inter-policy server launch is wrapped via `-EncodedCommand` for SSH+PowerShell reliability (commit `2b3cbea`).

A.6 Sections 7.4 and 7.5 agentic eval (`agent_bench.py`)

```
EVOKE_MODEL_PATH=$EVOKE_MODEL_PATH \
EVOKE_BUDGETS=512,1024,2048 \
  uv run python scripts/agent_bench.py
```

The script runs five strategies (`recency`, `streaming_llm`, `evoke_discard`, `evoke_breadcrumb`, `evoke_kv_restore`) at each budget. For the Section 7.5 attention row, set `EVOKE_W_ATTENTION=0.5` and `EVOKE_ATTN_LAYER=20`, which enables the `evoke_attention` strategy.

A.7 Section 7.6 keepalive workload (`agent_bench_keepalive.py`)

```
EVOKE_MODEL_PATH=$EVOKE_MODEL_PATH \
EVOKE_BUDGETS=512,1024,2048 \
  uv run python scripts/agent_bench_keepalive.py
```

The keepalive variant has each filler-file content reference the central `config.py` so the model's attention keeps coming back to the config block, which is what the attention scorer exploits.

A.8 Multi-session pool sanity check (`verify_multi_session.py`)

```
# Server must already be running (see A.4)
EVOKE_SERVER='http://localhost:8000' EVOKE_MODEL_NAME='qwen25' \
  uv run python scripts/verify_multi_session.py
```

The script drives two sessions with distinct planted codewords through interleaved chat requests and asserts that each session retrieves its own codeword, confirming state-swap correctness.

B The Fork

The llama.cpp fork is hosted at `github.com/Anyesh/llama.cpp` (mirror of the working tree at `/mnt/data/llama.cpp` during development). The fork tracks upstream `ggml-org/llama.cpp` and adds the EVOKE-specific commits listed below.

B.1 EVOKE-specific commits in the fork

Commit	Title	What it adds
`1f37e75e7`	EVOKE: iSWA dual-cache support in `kv_block_save/load` (#41)	`llama_get_kv_cache_swa()` and an extended save/load buffer layout that covers both base and SWA sub-caches on Gemma 3 and 4
`151f1cf93`	EVOKE: multi-layer attention capture (up to 16 layers per decode)	`llama_attn_capture_set_layers(ctx, layers, n)` writes one slab per layer in a single decode
`713f202e8`	EVOKE: attention-weight capture primitive	The original single-layer side-compute path: `llama_attn_capture_set_layer`, `_set_buffer`, `_get_dims`, `_get_written`
`9b6232446`	EVOKE: attention-only `seq_rm` and `seq_add` primitives	`llama_kv_block_seq_rm` and `llama_kv_block_seq_add` reach the attention sub-cache directly via `llama_get_kv_cache`, for "no-shift" eviction modes
`a29599f4c`	EVOKE: `kv_block` primitives now cover hybrid memory and mrope models	Extends `llama_get_kv_cache` to unwrap `llama_memory_hybrid` via `get_mem_attn()` and `llama_memory_hybrid_iswa` via `get_mem_attn()->get_base()`. Removes the mrope `seq_add()` and `get_can_shift()` guards.

B.2 The two recovery primitives, verbatim

From `include/llama.h`:

```
LLAMA_API size_t llama_kv_block_save(
    struct llama_context * ctx,
            llama_seq_id   seq_id,
               llama_pos   p0,
               llama_pos   p1,
                 uint8_t * dst,
                  size_t   cap);

// Load a buffer produced by llama_kv_block_save into seq_id starting at new_p0.
LLAMA_API bool llama_kv_block_load(
    struct llama_context * ctx,
             llama_seq_id   seq_id,
           const uint8_t * src,
                  size_t   size,
               llama_pos   new_p0);
```

`save` returns the number of bytes written (or zero on error, e.g. `cap` too small). `load` returns true on success. The caller is responsible for the buffer allocation in both directions.

B.3 Build instructions

The fork builds cleanly against the upstream llama.cpp toolchain. Requirements: CMake $\geq$ 3.14, a C++17 compiler, and (for the GPU path) CUDA Toolkit 11.8 or later (I tested with 13.1). For Windows, the Windows SDK is required in addition to MSVC; on a fresh chihiro-class machine without the SDK the build will fail on missing `winsdk` headers. A reference Windows build wrapper is in `scripts/build_fork_chihiro.ps1`.

```
cmake -B build -DLLAMA_CUDA=ON -DBUILD_SHARED_LIBS=ON
cmake --build build --config Release
```

The Python binding (`evoke/_engine_lib.py` and `evoke/llama_engine.py`) loads the shared library indicated by the `LLAMA_CPP_LIB` environment variable (a file path). It then derives `LLAMA_CPP_LIB_PATH` (a directory) for `llama-cpp-python` 0.3.x compatibility so a single loaded module backs both the Python wrapper and the EVOKE ctypes binding. Mixing two builds (the upstream wheel and the fork) causes layout-mismatch crashes; using a single fork build everywhere is the supported configuration.